Hackathon PS Verdict & Build Plan

Oct 9, 2026 · @Master Bruce

1. Executive verdict

Pick Project IO SIGNAL SUDOKU (industrial #02). Build a rule engine that compiles the user's rules into an OR-Tools CP-SAT model, allocates every signal with zero violations, turns infeasibility into explained TQs, and re-allocates Rev B with the fewest possible moves. It scores 87/100 on the weighted framework (section 5).

Second: T4-2 indoor positioning (81). A robust particle filter with NLOS gating, scored on positioning error. **Third: T4-3 sensor trust (77).** Cross-sensor consensus plus typed fault detection.

Why #1 wins on probability, not glamour. Every item in its definition of done (100% allocated or a TQ, zero violations, correct CHARM and names, unchanged signals stay put, no code change for a new rule; IND p6) can be checked by a script before the judges check it. Its hidden test (an unseen IO list plus a revision) is passed by construction by a correct solver, while greedy or LLM-placed allocations break on it. It needs no training data. And at this event's 2025 edition both top overall prizes went to sponsor-set tracks.

The main trade-off: you give up visual "AI wow" for provable correctness. A judge who equates AI with neural networks may undervalue a solver. Counter it by naming constraint programming as AI, adding a natural-language rule assistant that writes rules (never placements), and showing a live revision diff.

The biggest unresolved uncertainty: you have not yet seen the organisers' sample IO MDB, allocation template or rubric, and the provider (very likely Emerson) is inferred. **Stage 1 runs 10–11 October, i.e. tomorrow,** and registration was due "before 9 October" (VIT CSE-AI). Confirm registration tonight; the Stage 1 deck can be built from this doc.

2. Source-document audit

You attached three PDFs, not one merged file. I cite them as **IND** (industrial challenges, 8 pages, no sponsor branding), **IMO** (iMocha / CODE APEX 3.0, 9 pages) and **T4** (Track 4, Technology for Social Cause, powered by UST, 12 pages). Page numbers are PDF page order.

2.1 Requirements checklist per statement

#	Statement	What must be built	Hard "definition of done"	Data	Hardest part
1	T4-1 Cold-storage excursion early warning (T4 p3–5)	Model that predicts P(excursion) within a future window, separates normal recovery from real trouble, names the cause (door, ambient, cooling loss), dashboard or API	Early enough to act, false alarms "under control" (p5)	Organisers *may* supply synthetic series + hidden set; public data allowed (p5)	Lead time vs false-alarm trade-off on unseen fault mixes
2	T4-2 Indoor positioning, sparse anchors (T4 p6–8)	2-D position from range/RSSI, works with few anchors, missing data, NLOS bias; uncertainty; accuracy vs anchor density and noise	"Lowest practical positioning error" on an unseen test set (p8)	Fully synthetic; public sets for baselines (p8)	NLOS bias with 3–4 anchors, where geometry is already weak
3	T4-3 Sensor trust, drift, false alarms (T4 p9–11)	Detect spikes, stuck values, noise, drift; tell real events from sensor faults; trust score over time; maintenance flag; explanations	Decide which alarms to trust and the fault type (p11)	Fully synthetic + optional public IoT data (p11)	Slow drift vs a genuine slow environmental trend

#	Statement	What must be built	Hard "definition of done"	Data	Hardest part
4	IM-01 Vibe Coding (IMO p3–4)	Better candidate experience and/or skill report for iMocha's AI-assisted coding test	Prototype + one sample report + why it is fair (p3–4)	Public/sample/synthetic only (p9)	Measuring AI-collaboration skill fairly from a session log
5	IM-02 Autonomous AI QA engineer (IMO p5)	Agent that explores a web app, writes and runs tests, files reproducible bug reports, emits reusable tests	Finds a measurable share of planted bugs; tests re-run; every report reproducible (p5)	Team chooses or builds the sample app	Knowing what is a bug without a spec (the test oracle)
6	IM-03 Skill Passport (IMO p6)	Evidence-linked skill profile from repos/resume, role gap, learning plan, recruiter view	No skill shown as verified without evidence (p6)	Public/sample data	Grading skill *level* from code, not just detecting a skill
7	IM-04 AI Job Simulator (IMO p7)	AI characters in 2–3 workplace scenarios, adaptive, rubric debrief quoting the session	Every debrief point backed by a quoted moment (p7)	None needed	Rubric scoring that is consistent, not just plausible
8	IM-05 Codebase Mentor (IMO p8)	Repo Q&A with file/line citations, architecture map, guided tours, starter tasks, living docs	Every answer cites code; peer test shows faster onboarding (p8)	Any open-source repo	Citation correctness + running a credible user study

#	Statement	What must be built	Hard "definition of done"	Data	Hardest part
9	EAGLE EYE cabinet QC (IND p1–3)	Photo + GA drawing + wire/label lists in; PASS/FAIL checklist, boxed FAIL snippets, re-check history; plug-in checks	Finds missing/misplaced part, wrong label, wrong ferrule; **a correct item is never marked FAIL**; generalises to an unseen cabinet (p3)	Organiser samples, hackathon use only; hidden planted-fault photos (p3)	Reading tiny, curved ferrule text in a crowded photo with zero false FAILs
10	IO SIGNAL SUDOKU (IND p4–6)	Excel IO list in; every signal assigned cabinet → CIOC → baseplate → CHARM by user-editable rules; TQ list; fill report; Rev A vs Rev B with fewest moves	100% allocated or a TQ; zero rule violations; correct CHARM, terminal block and names; unchanged signals stay put; new rule needs no code change (p6)	Organiser sample MDB + template; hidden unseen MDB + revision (p6)	Minimum-move re-allocation under hard rules, plus explaining infeasibility
11	UNTANGLE P&ID simplifier (IND p7–8)	P&ID PDF + IO list in; simplified sketch with major equipment and every IO tag, correct flow, duty/standby as parallel lines; tag-check report	Complete, accurate connections and direction, readable, generic on an unseen P&ID (p8)	Organiser samples, hackathon use only (p8)	Tracing lines through crossings and valves in a P&ID of unknown format

2.2 Inconsistencies in the documents

- **T4-1 has the wrong problem statement (T4 p3).** The block headed "Problem Statement" under the cold-storage title is the indoor-positioning text, word for word. The title, background (p3), "system should" list (p4), core challenge and dataset (p5) all describe excursion prediction. Four of five blocks agree, so the intended scope is excursion prediction; get written confirmation from the organisers before Stage 1.

- **T4-2's problem statement sits under a "Dataset Reference/Creation" heading (T4 p6),** and its "Suggested technology" (p7) is copied from T4-1 (forecasting + anomaly detection + classification). Positioning is an estimation and filtering problem; treat that line as a copy error, not a hint.
- **T4 is labelled "Proposed" throughout.** Datasets are things organisers "may" provide (p2, p5, p12). No stage timeline, deliverable format or rubric weights exist; p12 lists six scoring dimensions without weights.
- **IM-02: multi-agent is "Optional" in the expected solution (IMO p5) but listed as a Stage 3 expectation** in the same page's table. Plan to show cooperating roles in the finale even if they are thin.
- **EAGLE EYE: wiring is tagged STRETCH (IND p2) but colour and terminal checks appear in the core Inspect and Compare features (p3).** Treat GA, label and ferrule as must-haves and wiring as stretch.
- **EAGLE EYE's "never marked FAIL" rule (p3) has no third state.** The output spec is PASS/FAIL only, yet a correct system must sometimes say "cannot read". You will need a NEEDS-REVIEW status and should ask whether it is allowed.
- **IO SUDOKU rule ambiguities (IND p5).** Whether a 6-, 12- or 2-pair cable may straddle two baseplates is not stated (the figure only shows a 24-pair cable on a boundary). A cable carrying both A and B signals cannot satisfy rules 2 and 4 at once, which must become a TQ. The baseplate pattern `<CIOC no>-CBP-<n>` gives `001-CBP-1`, while the CIOC pattern yields `AR1CIOC100001`, so "CIOC no" is undefined.
- **UNTANGLE counts disagree (IND p7).** The caption says 14 instruments are in the IO list; the IO-list snippet shows 3 rows "+ 10 more", which is 13.
- **No industrial document names its sponsor, event or timeline.** The hardware vocabulary points to Emerson (section 3), but that is an inference.

2.3 Who supplies the data

Statements	Supplied by organiser	You must source	Restriction
EAGLE EYE, IO SUDOKU, UNTANGLE	Sample inputs, templates, checklists; hidden test at judging	Anything for pre-event development	"Hackathon use only, do not share" (IND p3, p6, p8): never upload to public repos or third-party APIs without checking

Statements	Supplied by organiser	You must source	Restriction
T4-1, T4-2, T4-3	Possibly a synthetic dev set + hidden test (not committed)	Synthetic generator and public datasets	Use only legally reusable public data (T4 p2)
IM-01 to IM-05	Nothing	Sample apps, repos, sessions, resumes	Public, sample or synthetic only; no personal data without consent (IMO p9)

3. Hackathon and sponsor intelligence

All three PDFs belong to **VIT CODE APEX 3.0**, run by the Artificial Intelligence Students' Forum (AISF) and the Department of CSE (AI), Vishwakarma Institute of Technology, Pune. Stage 1 is online on 10–11 October 2026, which is tomorrow. (VIT CSE-AI events page)

3.1 Verified event facts

Item	Fact	Source
Format	3 stages: Stage 1 online evaluation 10–11 Oct; Stage 2 online evaluation 15–18 Oct; Stage 3 24-hour offline finale 24 Oct 2026, VIT Bibwewadi	VIT CSE-AI
What each stage wants (iMocha track)	Stage 1 idea + presentation; Stage 2 online prototype; Stage 3 complete build in the 24-hour finale; one idea carried through all three	IMO p2
Prizes	₹2,00,000+ pool: ₹50,000 / ₹30,000 / ₹20,000 overall, plus track-wise winners, internships, mentorship	VIT CSE-AI
Team and fee	1–4 members; ₹300 (non-VIT + GST); "register before 9 October"	VIT CSE-AI
Tracks named publicly	Track 03 Open Innovation; Tracks 01–02 "coming soon" when posted; Track 4 = UST social cause (T4 p1); iMocha track (IMO p1)	VIT page, PDFs

Item	Fact	Source
Judging rubric	**Not found publicly.** The Unstop listing (unstop.com/o/HhCvFaZ) did not render for me; Track 4 gives unweighted dimensions (T4 p12)	—
Previous editions	Code Verse, 24–25 Sep 2025, "under the banner of VIT Code Apex", 240+ teams; CODEAPEX 2.0, 27–28 Mar 2026 finale, ₹70,000 pool	VIT CSE-AI
UST's history here	UST was a goodies/in-kind sponsor of the 2025 edition	VIT CSE-AI

Action tonight: confirm your team is registered on Unstop and that Stage 1 asks for the same deliverable the iMocha PDF describes (idea + presentation).

3.2 Who owns each statement

Statements	Provider	Status
T4-1 to T4-3	UST ("powered by UST", T4 p1)	Verified
IM-01 to IM-05	iMocha (IMO p1–2)	Verified
EAGLE EYE, IO SUDOKU, UNTANGLE	Not named in the PDF. Very likely **Emerson** or an Emerson DeltaV engineering partner	Inference (below)

The industrial evidence: DeltaV, CHARMs and the CHARM I/O Card (CIOC) are Emerson's Electronic Marshalling product line (Emerson). The EAGLE EYE anatomy photo shows an Emerson SolaHD 40A power supply (IND p2). Emerson has a Pune-area plant at Talegaon (Zeal Polytechnic visit note). Two such signals are suggestive, not proof; ask a mentor who is judging this track.

3.3 What each provider is likely to reward

Provider	Verified signals	Evidence-based inference (what their engineer checks)	What would make them distrust you
Industrial (likely Emerson)	Every IND definition of done is machine-checkable: "zero violations", "100% allocated", "a correct item is never marked FAIL", hidden unseen test (IND p3, p6, p8). IO SUDOKU says "it's a constraint-packing puzzle" and "No DCS background needed" (p6). Emerson markets Electronic Marshalling on absorbing late changes without re-design (Emerson)	Correctness on their hidden file, revision handling, rules changeable without code, Excel output that drops into their workflow. A deterministic solver is welcome: the PDF calls it a puzzle, not an ML task	Any rule violation, a hand-edited output, a demo that only runs on the sample, an LLM that "usually" places signals right
iMocha	AI-powered skills-intelligence company; Vibe Coding "comes directly from a product iMocha is prototyping"; other four PS are agents that "back their results with evidence" (IMO p2). Products include a 3,000+ skill library, the patented AI-LogicBox and AI proctoring (iMocha)	Fairness, explainability and evidence links in any score. Competitors already sell AI-collaboration coding tests (CodeSignal on iMocha alternatives), so iMocha needs report and candidate-experience ideas, not another IDE	Scores an LLM invents without traceable evidence; "verified" skills with no source (IMO p6)

Provider	Verified signals	Evidence-based inference (what their engineer checks)	What would make them distrust you
UST (Track 4)	D3CODE judging covers originality, social impact, scalability and user experience (UST 2024 release). UST's CTO asks students to deliver measurable impact responsibly and at scale (D3CODE 2025 release). T4 p12 scores hidden-test performance, robustness, explainability, deployability, visualisation and clarity of assumptions	A leaderboard-style number on hidden data plus an honest limitations slide. Social framing (hospital asset tracking, vaccine cold chain) matters to UST's jury more than to Emerson's	Accuracy only on your own synthetic data; no stated assumptions

Marketing vs the judging engineer. Emerson markets AI and digital transformation, but the engineer who wrote IO SUDOKU defined success as zero violations and fewest moves. iMocha markets "AI-powered", but its PDF repeatedly asks for evidence and fairness. Optimise for the definition of done, then describe the result in the provider's vocabulary.

Strategic inference from this event's history. In the 2025 edition, the overall first and second prizes both went to sponsor-set tracks (DRDO and "Denitsu" challenges); the general track took only runner-up places (VIT CSE-AI). One edition is thin evidence, but it favours a sponsor-defined problem with a crisp success test.

4. Historical winners research

No winning project from this event, iMocha or the industrial provider is documented technically in public. What exists are winner names, judging structures and analogous research benchmarks; the lessons below come from those, labelled by evidence class.

4.1 What I found, by evidence class

Class	Event	Winner and project	What is documented	Lesson for us
1 · this event	VIT Code Verse, Sep 2025 (VIT Code Apex banner)	1st overall: Team Tech Titans, Track 1 DRDO Challenge. 2nd overall: Team Phoenix, Track 2 "Denitsu" Challenge. Track 3 general: runner-ups only (Farmops, Shunya) (VIT)	Names and tracks only; no project write-ups	Sponsor-set problems took both top prizes
1 · this event	CODEAPEX 2.0, Mar 2026	Not published on the department page	Stages only: screening → pitching → 24-hour build	Same 3-stage funnel as 3.0; Stage 1 is a screen, not a build
1 · UST-run	UST D3CODE 2025	Global: Neural Navigators (US), "PRISM: Predictive & Reporting Insights with Science & Models". India: Team Pulimada, College of Engineering Trivandrum (UST release)	Title only; no method or metrics	UST favours predictive + reporting systems with measurable impact

Class	Event	Winner and project	What is documented	Lesson for us
1 · UST-run	UST D3CODE 2024	Winners named in releases; judged on originality, social impact, scalability, UX (release)	Idea-phase PPT must state problem, solution, impact, resources and **named competitors** (HackerEarth guideline)	Name your baselines and rivals in the Stage 1 deck
1 · UST-run	UST D3CODE 2019 (first edition)	Participant write-up: UST handed out tightly specified problems and a scoring sheet with explicit pointers (SDSLabs blog)	Community-written, secondary	Expect rubric scoring: map every slide to a rubric line
2 · analogous	Emerson Innovation Challenge, Philippines	Student prototypes built over five months with Emerson mentors (Newsbytes)	Format differs (months, hardware)	Not evidence of what Emerson's Pune engineers pick
2 · analogous research	Wire Label Reader for control-cabinet assembly	Deep CNN reading wire markings, reported 99.7% recognition in production (Energies 2021)	Peer-reviewed; trained on a fixed camera rig, not phone photos	EAGLE EYE ferrule reading is solvable only with controlled imaging or lots of data
2 · analogous research	P&ID-to-graph benchmarks	Best reported real-world edge mAP is 63.8% (SynthPID on PID2Graph OPEN100) (arXiv 2604.16513)	Research SOTA, 2026	Raster P&ID tracing is unsolved research; do not bet a 24-hour build on it

Class	Event	Winner and project	What is documented	Lesson for us
2 · analogous research	LLM web-testing agents	Benchmarks of bug-injected apps show oracle reliability and long-horizon flakiness as the main failures (WebTestPilot, WebTestBench)	Peer-review status varies	An IM-02 agent wins by fixing the oracle, not by adding agents

Searches that returned nothing usable: "CODE APEX 3.0" on Unstop (page did not render), CODEAPEX 2.0 winners, iMocha-sponsored hackathon winners, "Signal Sudoku" / "Project Untangle" / "Project Eagle Eye" with Emerson. Class-3 "popular online" projects were excluded.

4.2 What the strongest documented results have in common

- **A number on held-out data.** UST's own jury language is "measurable impact"; Track 4 scores hidden-test performance first (T4 p12).
- **A defined scoring sheet.** UST has used explicit pointers since 2019; the industrial PDFs publish a checkable definition of done.
- **Sponsor-set problems over open ones** at this event in 2025.
- **Named baselines and competitors** in the idea round (D3CODE template).

How to beat a typical submission: most teams will show a polished UI and a claim. Show a baseline, your method, the delta, and a failure case you understand. In IO SUDOKU that means a rule-violation count of zero against a greedy baseline that violates rules, and a revision that moves fewer signals.

5. Comparison of all 11 problems

IO SIGNAL SUDOKU scores 87/100, ahead of T4-2 indoor positioning (81) and T4-3 sensor trust (77). The gap between #1 and #2 (6 points) is about the size of the uncertainty on each score, so section 7 says when #2 should win instead.

5.1 Weighted scores (criterion weight in the header; arithmetic in the last column)

Rank	Statement	Align /20	Feasible /20	Contribution /15	Measurable /15	Data /10	Robust /10	Sponsor /5	Demo /5	Total = sum
1	IO SIGNAL SUDOKU	18	18	12	14	8	9	4	4	18+18+12+14+8+9+4+4 = **87**
2	T4-2 Indoor positioning	17	17	10	15	8	7	3	4	17+17+10+15+8+7+3+4 = **81**
3	T4-3 Sensor trust	16	17	10	13	8	6	3	4	16+17+10+13+8+6+3+4 = **77**
4	IM-02 AI QA engineer	17	13	11	14	7	6	3	4	17+13+11+14+7+6+3+4 = **75**
5	T4-1 Cold-storage warning	13	17	10	13	7	6	3	4	13+17+10+13+7+6+3+4 = **73**
6	UNTANGLE P&ID	15	10	13	12	5	5	4	5	15+10+13+12+5+5+4+5 = **69**
7	IM-05 Codebase mentor	14	15	7	9	9	6	3	4	14+15+7+9+9+6+3+4 = **67**
8	EAGLE EYE cabinet QC	14	9	13	12	3	4	4	5	14+9+13+12+3+4+4+5 = **64**
9	IM-01 Vibe Coding	16	13	9	6	6	5	5	4	16+13+9+6+6+5+5+4 = **64**
10	IM-03 Skill Passport	14	15	7	6	7	5	5	4	14+15+7+6+7+5+5+4 = **63**
11	IM-04 Job Simulator	13	15	6	5	7	4	4	5	13+15+6+5+7+4+4+5 = **59**

EAGLE EYE and IM-01 tie at 64; EAGLE EYE ranks higher because its ceiling is higher if its data turns out good.

5.2 Ranking with risk, data and differentiation

Rank	Statement	Score (±)	Key technical challenge	Data readiness	Training difficulty	Implementation risk	Differentiation opportunity	Winning potential	Fact that would change the rank
1	IO SIGNAL SUDOKU	87 (±6)	Min-move re-allocation under hard rules; explaining infeasibility as TQs	Sample MDB after registration; synthetic generator is easy	None (solver)	Low–medium	High: solver + unsat-core TQs + revision diff vs greedy Excel macros	High	Judges insisting on ML, or a messy multi-file sample MDB
2	T4-2 Indoor positioning	81 (±7)	NLOS bias with 3–4 anchors	Synthetic + public UWB sets (section 6)	Light (NLOS classifier)	Low	Medium: robust particle filter + NLOS gating vs plain EKF	Medium–high	Organisers not releasing a dataset, or a scoring format we can't match

Rank	Statement	Score (±)	Key technical challenge	Data readiness	Training difficulty	Implementation risk	Differentiation opportunity	Winning potential	Fact that would change the rank
3	T4-3 Sensor trust	77 (±7)	Slow drift vs real slow trend	Synthetic + Intel Lab data	Light (GBM)	Low	Medium: cross-sensor consensus + typed faults	Medium	Hidden set with fault types outside the brief
4	IM-02 AI QA engineer	75 (±8)	Test oracle without a spec	Build own bug-seeded app	None–light (API LLM)	Medium	Medium: deterministic crawler + verified repro	Medium	Finale internet or API limits
5	T4-1 Cold storage	73 (±8)	Lead time vs false alarms	Synthetic	Light	Low	Medium: grey-box thermal model	Medium	Organiser fixes the copied problem text
6	UNTANGLE	69 (±10)	Line tracing through crossings	Organiser samples; public research sets	Heavy if raster	High if raster	High	Medium if vector PDF	**Sample PDFs are vector** (text and lines extractable) → ~74
7	IM-05 Codebase mentor	67 (±7)	Correct citations; user study	Any repo	None	Low	Low: crowded market	Low–medium	—
8	EAGLE EYE	64 (±10)	Reading ferrules with zero false FAILs on an unseen cabinet	Organiser photos only; no public set	Heavy	High	High	Low–medium	Sample photos with legible, flat ferrule text
9	IM-01 Vibe Coding	64 (±8)	Fair scoring of AI collaboration	Self-generated sessions	None	Medium	Medium	Medium (best sponsor fit)	iMocha engineers judging and rewarding product ideas
10	IM-03 Skill Passport	63 (±7)	Grading skill level, not presence	Public repos	None	Low	Low: LLM-wrapper risk	Low	—
11	IM-04 Job Simulator	59 (±7)	Consistent rubric scoring	None needed	None	Low	Low	Low	—

5.3 Why the scores fall where they do

- **IO SUDOKU:** every definition-of-done item is checkable by a script, so you can prove you meet it before the judges test you. It is the only statement whose hidden test (unseen MDB + revision) a correct deterministic engine passes by construction. Lost points: Demo 4/5 (spreadsheets demo poorly unless visualised), Sponsor 4/5 (owner inferred).
- **T4-2:** the cleanest objective metric of the 11 (positioning error on a hidden set), and well-understood estimators. Lost points: Contribution 10/15, since robust filtering is textbook and many CSE-AI teams can reach a good EKF.

- **T4-3:** similar profile; drift-vs-trend separation is the real problem and is harder to prove than positioning error.
- **IM-02:** measurable via a planted-bug benchmark (IMO p5), but live agents are flaky and the finale may restrict APIs.
- **T4-1:** loses 7 on Alignment because its problem statement is the wrong text (section 2.2).
- **UNTANGLE and EAGLE EYE:** high ceilings, but research-grade perception on organiser-only data in 24 hours; their scores hinge on facts you only learn after registration.
- **iMocha GenAI statements (IM-01, 03, 04, 05):** outcomes judged largely subjectively, and the easiest place for 50 near-identical LLM wrappers to compete.

6. Detailed investigation of the top three

Detail is weighted toward #1. Each finalist gets its data, method, evaluation and risks; the full build spec for #1 is in section 8.

6.1 IO SIGNAL SUDOKU — constraint solver with minimum-move revisions

Problem and difficulty. Assign every signal to cabinet → CIOC → baseplate → CHARM slot. Capacities: 12 CHARMs per baseplate, 8 baseplates per CIOC, 4 CIOCs per cabinet, so 384 slots per cabinet (IND p5). The hard parts are not the packing itself but (a) a Rev B re-allocation that moves almost nothing, (b) explaining why something cannot be placed, and (c) rule semantics the PDF leaves open (section 2.2).

Core modelling insight. The unit of decision is the **cable**, not the signal. Rule 4 forces every 24/12/6/2-pair cable to land contiguously in pair order, so a cable is a fixed-length block of slots (spare pairs included: a 24-pair cable fills exactly 2 baseplates, p5). That collapses about 10,000 signals into roughly 1,000 blocks, assuming an average of 10 used pairs per cable, and makes the problem a 1-D interval packing per CIOC.

Data.

Source	Access	Use	Limits
Organiser sample IO MDB + allocation template	After registration; "hackathon use only, do not share" (p6)	Ground truth for formats, columns, naming	Do not paste rows into cloud LLMs or public repos

Source	Access	Use	Limits
Public IO lists	None found; real MDBs are proprietary	—	—
Our synthetic generator	Built by us	Scale tests (200–12,000 signals), adversarial cases, Rev B pairs	Must be tuned to the sample's statistics

Synthetic generator spec: signal count N; IO-type mix (start at AI 35%, DI 35%, DO 20%, AO 10% as an assumption, then match the sample); cable-size histogram over {24, 12, 6, 2} with spare pairs; groups with sizes drawn from a long-tailed distribution; an A/B-segregated fraction; DO subtypes (SOV, horn, beacon → isolated CHARM). Inject defects with known labels: blank IO type, duplicate tag, pair number above cable size, A and B in one cable, a group bigger than a cabinet. Rev B generator: add 3–10%, delete 1–5%, change type on 1–3%, re-cable 1–2%. Validate realism by comparing type mix, cable-size histogram and group-size distribution against the sample. There is no learned model, so "learning generator artifacts" is not a risk; under-covering real messiness is, so also fuzz headers (renamed, reordered, extra junk rows, multiple files).

Trainability. No training. CP-SAT runs on a laptop CPU. An LLM is optional and only for turning a typed sentence into a rule definition, never for placing signals.

Prior art and baselines. Today's baseline is hand allocation in Excel (p4). I found no public tool doing rule-driven CHARM allocation with minimum-move revisions; Emerson's marketing stresses that Electronic Marshalling absorbs late changes in hardware, not that the allocation spreadsheet does (Emerson). Ask the provider whether an internal tool exists. The strongest baseline a rival team will build is **greedy first-fit by group and cable**; ours must beat it on violations, cabinets and moves. Honest label: this is an engineering contribution combining known constraint-programming techniques (interval packing, unsatisfiable cores, minimal perturbation), not a new algorithm.

Winning thesis.

Element	Statement
Problem	Hand allocation of 10,000+ signals is slow, and every customer revision forces rework and impact tracing (p4)

Element	Statement
User	DeltaV project engineer preparing IO allocation and answering customer revisions
Core insight	Compile rules into constraints over cable blocks; verify output with an independent checker; treat Rev B as "stay close to Rev A"
Technical contribution	Rule DSL → CP-SAT model; unsat-core TQs; lexicographic objective (feasible → fewest moves → fewest cabinets)
Measurable advantage	0 violations on every test file (greedy will not), fewer moved signals on Rev B than a from-scratch re-solve, cabinets within a stated lower bound
Live demo	Load unseen MDB → allocate → checker shows 0 violations → load Rev B → impact report with N moves → change spare 10%→20% in the rule panel → re-run with no code change
Differentiation	Proof of correctness (checker) and explained TQs, where most teams will show only a filled sheet
Failure condition	Any checker violation on a valid input, or Rev B moving signals the rules did not force to move

MVP: CSV/Excel in, cable-level greedy + CP-SAT feasibility, independent checker, filled template out. Differentiated: unsat-core TQs, minimum-move Rev B, rule panel with saved rule sets. Stretch: natural-language rule assistant, cabinet-grid visualisation, multi-file MDB merge.

Architecture.

```
flowchart LR
  A[IO MDB Excel files] --> B[Column auto-mapper\nfuzzy headers + value
sniffing]
  B --> C[Data validator\ndata TQs]
  R[Rule panel\nYAML rule sets] --> D[Rule compiler]
  C --> E[Cable-block builder]
  E --> F[CP-SAT allocator\nstage A: cabinet/CIOC\nstage B: slot intervals]
  D --> F
  P[Rev A allocation] -->|hints + move penalty| F
  F -->|infeasible| G[Core extractor\nrule-conflict TQs]
  F -->|solution| H[Name generator\npattern templates]
  H --> I[Independent checker]
  D --> I
  I --> J[Template writer\nopenpyxl into a copy]
  I --> K[Reports: fill %, spares,\nrule-check, TQ list, Rev diff]
  G --> K
  J --> L[(SQLite history\nrule sets, revisions)]
```

Algorithms. Each cable c has length s_c (pairs including spares). Slots are numbered on one line per CIOC (96 slots = 8 × 12). A Boolean y_{c,k} puts cable c on CIOC k and an interval [st_c, st_c + s_c) places it.

```
\sum_k y_{c,k} = 1 \quad \forall c \qquad \text{NoOverlap}_k\{[st_c,
st_c+s_c) : y_{c,k}=1\} \qquad (st_c \bmod 12) + s_c \le 12 \ \ (s_c \le 12)
\qquad st_c \bmod 12 = 0 \ \ (s_c = 24)
```

- **A/B segregation:** per CIOC Booleans hasA_k ≥ y_{a,k} and hasB_k ≥ y_{b,k}, with hasA_k + hasB_k ≤ 1. This is linear in cables, not quadratic in pairs.
- **Group cohesion:** group g picks one cabinet; y_{c,k} implies cabinet(k) = cabinet(g). "In sequence" is a soft objective term, because a hard version can make valid inputs infeasible.
- **Spare room:** per cabinet, used baseplates ≤ ⌊32 × (1 − spare%)⌋.
- **CHARM and terminal block:** a lookup from signal type (rule 6 table), validated, never solved.
- **Objective, lexicographic:** (1) moves from Rev A, (2) cabinets used, (3) group-sequence breaks. Rev A positions are passed as solution hints.
- **Scale:** solve stage A (cables → CIOCs) and stage B (slot intervals per CIOC) separately; stage B splits into small independent models. Start from a greedy solution as a hint and keep a time limit. Engineering target, not a measurement: under 60 s for 10,000 signals

on a laptop.

- **TQs:** every rule instance gets an enable literal. On infeasibility, a separate single-thread feasibility solve with those literals as assumptions returns a conflicting subset (CPMpy on unsat cores); shrink it with deletion-based MUS, then render it in words ("Cable CBL-207 carries both A and B signals: rules 2 and 4 conflict"). OR-Tools users report assumption cores misbehave with objectives or parallel search (OR-Tools discussion), hence the separate solve.
- **Checker:** a separate module re-derives every rule from the output Excel. It is the evidence for "zero violations" and should share no code with the solver.
- **Plug-in rules:** each rule class has `compile(model)` and `check(allocation)`. New *instances* (another group rule, a different spare %) need no code; a generic "same/different/at most k by attribute at level" class covers most new rule types. Brand-new semantics still need a class; say so.

Stack.

Component	Choice	Why	Required?
Language	Python 3.11	"Python preferred" (p6)	Yes
Solver	OR-Tools CP-SAT	Interval and NoOverlap constraints, hints, assumption cores	Yes
Excel I/O	pandas + openpyxl, writing into a copy of the template	Preserves their formatting	Yes
Column mapping	rapidfuzz + synonym table	Auto-map with no network	Yes
Rules	pydantic models + YAML files	Typed, saveable rule sets	Yes
UI	Streamlit	Simple UI in hours	Yes
Storage	SQLite	Revisions and rule sets	Yes
Tests	pytest + hypothesis	Property tests: any valid input → 0 checker violations	Yes
Visuals	Plotly cabinet grid	Demo clarity	Optional

Component	Choice	Why	Required?
Rule assistant	Local LLM via Ollama, or an API on rule text only	"AI" framing without sending data	Optional

Fallback if CP-SAT stalls: greedy allocation passed through the same checker, with violations turned into TQs, never hidden.

Evaluation protocol.

Metric	Computed as	Strong result	Hides on its own
Coverage	(placed + TQ'd signals) / all signals	100%	Junk TQs that dodge placement
Violations	Checker count over all rules	0 on every file	Rules the checker doesn't encode
TQ precision / recall	Against injected defects with known labels	Recall 100%, precision as high as possible	Wording quality
Revision moves	Unchanged-in-B signals whose slot changed	0 unless a rule forces it	Quality of placing new signals
Cabinet efficiency	Cabinets used / lower bound, where lower bound = ⌈Σ s_c ÷ (384 × (1 − spare%))⌉	⩽ 1.05	Group-cohesion cost
Runtime	Wall-clock by N	< 60 s at 10,000 (target)	Variance across seeds
Determinism	Same input → same output hash	100%	—

Compare B0 greedy in file order, B1 greedy grouped and A/B-aware, and ours. Ablations: no min-move objective (re-solve Rev B from scratch), no hints, no decomposition. Test suite: 50 synthetic MDBs from 200 to 12,000 signals, 20 adversarial files, 10 Rev pairs, plus the organiser sample, none used for tuning. For the hidden test, prepare for unseen headers, new groups and a revision; do not try to guess its content.

Run	Violations	Cabinets / LB	Rev B moves	Runtime (s)
B0 greedy, file order	to measure	to measure	to measure	to measure
B1 greedy, grouped + A/B	to measure	to measure	to measure	to measure
Ours, full	to measure	to measure	to measure	to measure
Ours, no min-move	to measure	to measure	to measure	to measure

Risk register.

Risk	Prob.	Impact	Earliest warning	Mitigation	Fallback	Go/no-go
Rule semantics misread (sequence, boundary crossing)	High	Medium	Sample allocation contradicts our reading	Make each ambiguity a rule parameter; ask in Stage 1	Ship both modes	Before Stage 2
CP-SAT too slow at 10,000	Medium	High	> 60 s on 5,000 synthetic	Decompose, hint, time-limit	Greedy + checker + TQs	End of day 2
Messy or multi-file MDB	Medium	High	Sample has merged cells or several sheets	Mapper + validation TQs + mapping confirm screen	Manual mapping profile saved per project	Stage 2
Template writing breaks formatting	Medium	Medium	Diff of template vs output	Write into a copy cell by cell	CSV + template side by side	Stage 2
Judges want visible ML	Medium	Medium	Mentor feedback	Frame CP as AI; NL rule assistant	—	Stage 1 feedback
Data-sharing breach via cloud LLM	Low	High	Any API call carrying rows	LLM sees rule text only	Local model	Always
A rival also uses a solver	Low–medium	Medium	Stage 2 demos	Lead on revision moves + TQ explanations + checker	—	—

Skeptical judge review.

1. "This isn't AI." Constraint programming is classical AI, and the PDF calls the task a constraint-packing puzzle (p6). Show the NL rule assistant as the generative layer.

2. "It only works on your synthetic data." Show the organiser sample and 20 adversarial files, all with 0 checker violations.
3. "Your checker shares your bugs." It is a separate module with property tests; demo it catching a violation planted by hand in a copy.
4. "Rev B could move fewer signals." The move count is the solver's first objective; show the from-scratch ablation's count beside ours.
5. "New rule types need code." True for new semantics; new instances and the generic attribute rule do not. State it on the limits slide.

Subjective planning estimate: about 80% chance of a complete, working build under the assumptions in section 8.

6.2 T4-2 Indoor positioning — NLOS-aware particle filter

Problem and difficulty. Estimate 2-D position from ranges and/or RSSI with few anchors, noise, packet loss and non-line-of-sight (NLOS) bias, then report uncertainty (T4 p7–8). NLOS adds a positive bias to a range, and with only 3–4 anchors one biased range moves the solution a long way, because the geometry has no redundancy to outvote it.

Data.

Dataset	Access and licence	Contents	Fit
Organiser synthetic set	"May" be provided, with a hidden test (T4 p2, p8)	Anchor coordinates, range/RSSI, timestamps, ground truth	The scoring distribution; get its schema early
Bregar UWB LOS/NLOS set	GitHub; check the repo licence	42,000 samples (21,000 LOS / 21,000 NLOS) from 7 indoor locations, with CIR features	NLOS **classification** only: no reference positions, per its README
Bregar UWB Positioning and Tracking	Zenodo v2.0, Aug 2023, CC BY 4.0 (v1 was CC BY-NC-SA)	Ranges from 4 indoor environments along walking paths with NLOS labels	Real-data validation of the estimator

Dataset	Access and licence	Contents	Fit
Flueratoru et al. UWB ranging	Zenodo, 2022	True vs measured distance, CIR and LOS/NLOS label on 3 platforms	Calibrating the NLOS bias distribution
Ghent/imec industrial UWB	IEEE DataPort (account may be needed)	8 anchors, TWR and TDOA, experiments with NLOS links	Optional second real-data check

Important: if the organiser data carries only range and RSSI, CIR-based NLOS classifiers trained on the public sets will not transfer. Plan NLOS detection from residuals and temporal consistency, and use public CIR data only to shape the bias model.

Synthetic generator spec: floor plans of 20×15 m to 40×25 m with wall polygons; 3–8 anchors; random-waypoint and corridor trajectories at 0.5–1.5 m/s; range = distance + NLOS bias + Gaussian noise (σ 0.05–0.3 m); NLOS bias exponential with mean 0.3–1.5 m whenever the anchor–tag line crosses a wall; packet loss 5–30%; anchor drop-out bursts; RSSI by log-distance path loss with exponent 1.8–3.5 and shadowing σ 4–8 dB. Hold out whole floor plans and noise levels for testing, mirroring the hidden set's "different paths, noise levels and obstruction patterns" (p8).

Trainability. Light. The estimator is model-based; only a small gradient-boosted NLOS detector is trained, on per-anchor features (range residual against the predicted range, innovation size, rate of change, RSSI–range disagreement). CPU-only, minutes.

Method. Baselines: B0 linear least-squares multilateration; B1 Gauss-Newton + constant-velocity EKF. Ours: a particle filter (1,000–5,000 particles) whose per-anchor likelihood is a mixture of a LOS Gaussian and a positively skewed NLOS component, weighted by the detector's NLOS probability, plus optional wall constraints. Missing anchors simply drop out of the likelihood. Uncertainty = particle covariance, drawn as a 95% ellipse, and checked for calibration (truth inside the ellipse about 95% of the time). Honest label: robust filtering is textbook; the contribution is the NLOS-gated likelihood and a rigorous density-vs-noise study, which T4 asks for (p7).

```
flowchart LR
  A[Range / RSSI stream] --> B[Validator\nmissing, stale, outliers]
  B --> C[Per-anchor features]
  C --> D[NLOS detector\nGBM probability]
  B --> E[Particle filter\nmotion model]
  D --> E
  F[Floor plan + anchors] --> E
  E --> G[Position + 95% ellipse]
  G --> H[API + live map]
  G --> I[Evaluator\nerror CDF by anchors and noise]
```

Evaluation. Mean, median and p90 error (m); RMSE; error vs anchor count (3, 4, 6, 8) and noise level; ellipse coverage; update latency. Ablations: no NLOS gating, no motion model, EKF instead of PF. Strong result: a clear p90 gap over B1 at 3–4 anchors with NLOS, and calibrated ellipses; set the numeric target only after B1 is measured.

Demo and pitch. A hospital floor with 4 anchors; a wall goes NLOS mid-path; EKF jumps through the wall, the PF stays on the corridor and widens its ellipse. End on the error-vs-anchor-count chart.

Risks. Organisers release no data or a different schema (fallback: our generator plus the Zenodo set); a rival reaches similar error with a well-tuned EKF (lead on p90 and calibration, not the mean); RSSI-only data makes everything worse (show the RSSI curve honestly). Subjective completion estimate: about 85%; winning depends more on rivals than on our build.

6.3 T4-3 Sensor trust — cross-sensor consensus with typed fault detectors

Problem and difficulty. Decide which alarms to trust and name the fault type (spike, stuck, noise, drift, gain, dropout) when real environmental events also move the readings (T4 p10–11). The hard case is slow drift during a genuine slow trend, and single-sensor deployments where no neighbour can vote.

Data.

Dataset	Access	Contents	Fit
Organiser synthetic set	"May" be provided, hidden test with unseen fault combinations (p11)	Multi-channel series with labelled faults and events	The scoring distribution

Dataset	Access	Contents	Fit
Intel Berkeley Lab data	Public download, no warranty	54 sensors, about 2.3 million readings every 31 s, 28 Feb–5 Apr 2004, with x-y positions; has missing epochs	Realistic correlated base signals to inject faults into
Sensor-fault benchmark datasets (2016)	Paper describes tool and data	5,783,504 points with injected random, malfunction, bias, drift and polynomial-drift faults	Extra labelled test cases

Sharma, Golubchik and Govindan found that estimation methods using neighbour correlation detect faults accurately but cannot classify them, while learning methods can classify but are cumbersome to train (ACM TOSN 2010). That points directly at a hybrid: consensus to detect, a classifier to type.

Method.

1. For each sensor, estimate the "true" value from neighbours: a correlation-weighted median of the k most related channels.
2. Residual r_i(t) = reading − consensus. A real event moves the consensus, so residuals stay small; a fault grows one sensor's residual.
3. Typed detectors on the residual and raw signal: Hampel filter (spike), run-length of identical values (stuck), rolling MAD against neighbours (noise), CUSUM / Page–Hinkley plus a residual trend fit (drift, offset), slope of reading vs consensus (gain), missingness rate (dropout).
4. A LightGBM classifier types the fault from detector outputs; SHAP or the detectors' own values give the explanation.
5. Trust score per sensor = smoothed evidence from all detectors, with a recalibrate / inspect / replace threshold.

Single-sensor channels fall back to physical bounds and a temporal model, and the UI says confidence is lower there.

```
flowchart LR
  A[Sensor streams] --> B[Alignment + gap handling]
  B --> C[Neighbour consensus]
  C --> D[Residuals]
  D --> E[Typed detectors]
  B --> E
  E --> F[Fault classifier]
  E --> G[Trust score]
  F --> H[Alarm triage\ntrusted vs suspect]
  G --> H
  H --> I[Dashboard + explanation]
```

Evaluation. Event-level precision and recall for "trusted alarm"; fault-type macro-F1; false alarms per sensor-day; drift detection delay; trust-score AUROC. Baselines: raw threshold alarms, per-sensor z-score, Isolation Forest. Ablation: no consensus (single-sensor features only). Test on fault combinations held out from training, as the hidden set will be (p11).

Risks. Hidden faults outside the brief's list (keep an "unknown abnormality" class); correlated failures, such as a shared power supply, that look like real events (flag when all residuals jump together with voltage); and the classifier learning our injector's signature (train on one injector, test on the 2016 benchmark faults). Subjective completion estimate: about 85%.

7. Head-to-head comparison of the finalists

IO SUDOKU leads on hidden-test robustness and provability; indoor positioning leads on a single objective metric and on data you can start with tonight.

Dimension	IO SIGNAL SUDOKU	T4-2 Indoor positioning	T4-3 Sensor trust
Feasibility in 2 weeks	High: no training; known solver	High: textbook estimators	High
Novelty (honest)	Engineering: rules → CP-SAT, unsat-core TQs, min-move revision	Engineering: NLOS-gated PF + density study	Engineering: consensus + typed detectors

Dimension	IO SIGNAL SUDOKU	T4-2 Indoor positioning	T4-3 Sensor trust
Data readiness today	Low until the sample arrives; generator covers the gap	High: generator + three public UWB sets	High: generator + Intel Lab data
Model risk	None (deterministic)	Low (small classifier)	Medium (classifier may learn injector signature)
Engineering work	Medium–high: Excel template, rule panel, revision history	Medium: filter, API, map	Medium: detectors, dashboard
Measurable performance	Binary and absolute: 0 violations, 100% coverage, moves	Continuous: metres of error	Continuous: F1, false alarms
Sponsor fit	Strong: solves the exact pain in the PDF (revisions)	Medium: UST values impact + UX	Medium
Demo reliability	High: deterministic, offline	High	High
Hidden-test robustness	Very high if mapping holds	Medium: noise model may differ	Medium: unseen fault combos
Likely rival count	Lower: domain vocabulary scares teams off (inference)	Higher: classic ML track	Higher

When #2 should replace #1:

- Your team is ML-strong and has nobody comfortable with Excel automation and optimisation modelling.
- Organisers release the Track 4 dataset with a leaderboard before Stage 2; an objective ranking you can climb beats a rubric you can't see.
- Mentors at Stage 1 say the industrial judges want visible ML, or the industrial sample stays locked until the finale.
- The go/no-go in section 9 fails: CP-SAT can't reach a checker-clean 10,000-signal allocation and greedy is no worse.

When #3 should replace #2: the organisers release sensor data but not positioning data, or a teammate has done time-series anomaly work before.

When an outsider jumps the list: UNTANGLE becomes a top-3 pick if the sample P&IDs are vector PDFs (lines and tags extractable, no vision needed). EAGLE EYE becomes viable only if sample photos show flat, legible ferrule text and the hidden cabinet is the same model.

8. Recommended project specification

Build **CHARMSAT: rule-compiled DeltaV IO allocation with minimum-move revisions**, with the solver working by 15 October so Stage 2 shows a real allocator, not a mock-up.

Abstract. CHARMSAT reads a customer IO list from one or more Excel files, maps its columns automatically, validates the data, and compiles a user-editable rule set into a constraint model over cable blocks. OR-Tools CP-SAT assigns every signal to a cabinet, CIOC, baseplate and CHARM slot with zero rule violations, picks CHARM and terminal-block types, and generates names from user patterns. When rules or data conflict, it extracts the conflicting rules and raises a plain-language TQ. For a revision, it re-solves with "don't move what didn't change" as the first objective and reports added, deleted and moved signals and the cabinets affected. An independent checker re-verifies every rule on the exported template.

Scope.

Included	Excluded
Upload of one or many MDB files; auto column mapping with a confirm screen	Other DCS families (PDF says Phase 1 is DeltaV, p5)
Rule panel: add, edit, enable/disable, save rule sets per project	Wiring diagrams, loop drawings
Allocation → filled organiser template	Electrical load or heat calculations
TQ list (data, rule-conflict, capacity) and rule-check report	Any LLM placing signals
Cabinet/CIOC counts, fill %, spares per cabinet	Cloud deployment

Included	Excluded
Rev A vs Rev B impact report with move counts	
Optional natural-language rule assistant (rule text only)	

Architecture, algorithms and stack: as specified in section 6.1.

Dataset plan.

1. Build the synthetic generator and 50-file test suite before the organiser sample arrives.
2. When the sample arrives, profile it (columns, type mix, cable sizes, groups) and re-tune the generator to match.
3. Keep the sample local; never commit it to a public repo or send rows to a cloud API.
4. Freeze a held-out set (10 synthetic files + 5 adversarial + 3 revision pairs) that nobody tunes against.

Core technical contribution: provably rule-clean allocations, with explained infeasibility and minimum-move revisions, from rules that are data rather than code.

Acceptance criteria.

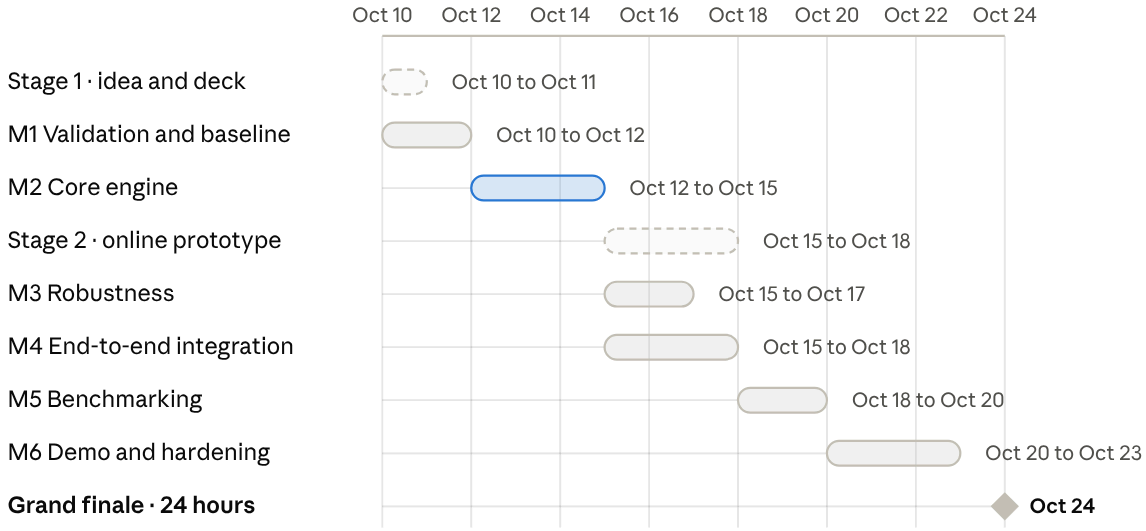
- [ ] 100% of signals placed or covered by a TQ on every test file
- [ ] 0 checker violations on every valid input, including the organiser sample
- [ ] Every injected data defect raises a TQ (recall 100% on the synthetic suite)
- [ ] Rev B moves 0 unchanged signals unless a rule forces it, and says which rule
- [ ] Changing spare %, a group rule or a naming pattern needs no code change
- [ ] 10,000-signal file allocates in under 60 s on a team laptop (target)
- [ ] Output opens in Excel with the template's formatting intact
- [ ] Same input produces the identical output file

Experimental plan: the baselines, ablations, metrics and benchmark table in section 6.1, run by one script (`make bench`) that writes a CSV and the charts used in the pitch.

Milestones.

The solver works by 15 Oct, so Stage 2 shows a real allocator

CHARMSAT plan, Oct 2026 · stage dates from VIT, milestones are our plan · dashed = judged stage



Stage dates: VIT CSE-AI events page · milestones: this plan

Milestone	Tasks	Deliverable	Depends on	Person-hours	Acceptance test	Fallback
M1 Validation and baseline	Generator v0; greedy B1; checker v0; 300-signal CP-SAT prototype	Baseline numbers on 20 files	—	18	Checker flags a hand-planted violation	—
M2 Core engine	Cable blocks; stage A/B model; rule compiler for rules 1–6; naming	Allocation CSV from synthetic files	M1	36	0 violations on 50 files; 10k signals < 60 s	Greedy + checker + TQs
M3 Robustness	Header fuzzing; defect TQs; unsat-core TQs; adversarial suite	TQ report	M2	24	Recall 100% on injected defects	Data TQs only
M4 End-to-end integration	Streamlit UI, rule panel, template writer, SQLite history, Rev diff	Clickable tool, Excel out	M2	24	Fresh file → template with no manual step	CSV beside template
M5 Benchmarking	Full bench, ablations, charts	Benchmark table	M3, M4	15	Script reproduces numbers from a clean clone	—

Milestone	Tasks	Deliverable	Depends on	Person-hours	Acceptance test	Fallback
M6 Demo and hardening	Demo data, backup video, README, offline install	Pitch + repo	M5	18	Two full dry runs under 5 min	Recorded demo

Person-hours: 18 + 36 + 24 + 24 + 15 + 18 = 135. Capacity assumption: 3 people × 14 days (10–23 Oct) × 4 h/day = 168. Buffer: 168 − 135 = 33 h, i.e. 33 ÷ 168 = 19.6%. With 4 people at the same rate, capacity is 4 × 14 × 4 = 224 h and the buffer rises to 89 h.

Finale plan. The iMocha PDF says teams carry and grow the same build across stages (IMO p2), but no published CODE APEX rule says whether prebuilt code may enter the finale. Ask in Stage 1.

Finale hours	If prebuilt code is allowed	If the build must start at hour 0
0–2	Run the hidden/unseen MDB; fix mapping	Environment, load sample, write rule YAML
2–8	Fix any checker failures; extra rules mentors ask for	Cable blocks + CP-SAT (A); validator + checker (B); UI shell + template writer (C)
8–12	Rev B on the finale revision	Integrate; first checker-clean run
12–18	Re-run benchmark; polish visuals	Min-move Rev B; TQ cores
18–22	Freeze at hour 20; rehearse	Adversarial tests; benchmark; freeze at hour 22
22–24	Backup recording; final dry run	Pitch rehearsal; backup recording

Responsibilities (3 people; a 4th if available).

Owner	Area	Owns
A	Solver and rules	Cable-block model, rule compiler, objectives, unsat-core TQs
B	Data and verification	Column mapper, validator, synthetic generator, independent checker, template writer

Owner	Area	Owns
C	Product and evidence	Streamlit UI, cabinet grid, revision report, benchmark harness, pitch
D (optional)	Adversary and assistant	Adversarial test files, NL rule assistant, revision history

Interfaces, frozen on day 1 so all three work in parallel:

```
@dataclass
class Signal: tag: str; io_type: str; subtype: str | None; cable: str; pair:
int; group: str | None; seg: str | None
@dataclass
class Cable: id: str; size: int; signals: list[Signal]; group: str | None;
segs: set[str]
class Rule(Protocol):
    id: str; enabled: bool
    def compile(self, model: "AllocModel") -> list["BoolVar"]: ...  # returns
enable literals
    def check(self, alloc: "Allocation", mdb: list[Signal]) ->
list["Violation"]: ...
# Allocation row: tag, cabinet, cioc, baseplate, slot, charm, terminal_block,
cabinet_name, cioc_name, baseplate_name
# TQ: id, kind ('data' | 'rule_conflict' | 'capacity'), items, rule_ids,
message
def allocate(mdb, rules, prev: "Allocation | None" = None, time_limit_s: int
= 60) -> "AllocResult": ...
def check(alloc, rules, mdb) -> list["Violation"]: ...
```

Deployment: runs offline on one laptop (`pip install -r requirements.txt` , `streamlit run app.py`), with a Docker image as backup. No internet is needed for the core flow.

Demonstration scenario.

1. Load an unseen IO list with renamed headers; show the mapping screen guess every column.
2. Allocate; show the cabinet grid, fill %, spares, and the checker at 0 violations.
3. Run greedy B1 on the same file; show its violations or extra cabinets in red.
4. Load Rev B; show the impact report and the moved-signal count beside a from-scratch re-solve.

5. Change spare room from 10% to 20% in the rule panel; re-run with no code change; cabinet count updates.
6. Load an adversarial file with A and B signals on one cable; show the TQ naming rules 2 and 4.

Pitch segment	Time	Speaker
Problem and stakes: 10,000 signals, every revision redone by hand	0:00–0:40	A
Why Excel and greedy fail: violations, churn on revisions	0:40–1:20	B
Insight: rules compile to constraints; an independent checker proves them	1:20–2:00	A
Live demo, steps 1–6	2:00–3:30	C drives, A narrates
Numbers: benchmark table vs baselines	3:30–4:20	B
Limits and next steps (new rule semantics need code; DeltaV only)	4:20–5:00	A

9. First 72 hours of work

The problem choice locks when you submit Stage 1 ("one idea, three stages", IMO p2), so the feasibility test has to run **before** that submission on Saturday, not after. The window below runs from Friday 9 Oct, 21:30 to Monday 12 Oct, 21:30.

Highest-risk question for the first 10% of time: can a cable-block CP-SAT model produce checker-clean allocations at 10,000-signal scale fast enough, and does the min-move objective clearly beat a from-scratch re-solve? A close second: does our reading of the ambiguous rules match the organisers'?

When	Owner	Task	Evidence before moving on
Fri 21:30–22:30	All	Confirm Unstop registration and track; read the Stage 1 submission format on the dashboard	Confirmation screenshot; deadline time noted

When	Owner	Task	Evidence before moving on
Fri 22:30–01:00	A	Draft Stage 1 deck from sections 1, 6.1 and 8: problem, insight, architecture, rule DSL, evaluation plan, named baselines and rivals	Deck outline
Fri 22:30–01:00	B	Send the organiser questions below	Message sent
Fri 22:30–01:00	C	Repo skeleton; install ortools, openpyxl, pandas, rapidfuzz, pydantic, streamlit, pytest, hypothesis	`pytest` runs green on an empty suite
Sat 09:00–13:00	B	Synthetic generator v0 (300–10,000 signals, labelled defects)	20 files on disk
Sat 09:00–13:00	A	**Experiment E1:** CP-SAT cable-block model for rules 2, 3, 4 on a 1,000-signal file	Allocation CSV
Sat 09:00–13:00	C	Checker v0 for rules 2, 3, 4; greedy baseline B1	Checker flags a hand-planted violation
Sat 13:00	All	**Gate 1 (choose the problem):** E1 is checker-clean in under 30 s → submit IO SUDOKU. Otherwise submit T4-2 using section 6.2	Decision recorded
Sat 13:00–20:00	A, C	Finish and submit the Stage 1 deck; 3 mock screens from Streamlit stubs	Submission receipt
Sun (Stage 1 slot)	A	Present Stage 1; note every judge question about rules and "AI"	Notes
Sun rest of day	A	**Experiment E2:** scale to 1,000 / 5,000 / 10,000 signals; record runtime and violations	Runtime table
Sun rest of day	B, C	**Experiment E3:** 10 Rev B pairs (5% changes); moves for min-move vs from-scratch; B1 vs ours on 20 adversarial files	Moves and violations table

When	Owner	Task	Evidence before moving on
Mon 18:00–21:00	All	Freeze interfaces (section 8); profile the organiser sample if released; download the Zenodo UWB set as a hedge	Interfaces doc committed
Mon 21:00	All	**Gate 2 (choose the method)**	Decision recorded

Gate 2 criteria.

- **Go:** E2 reaches a checker-clean 10,000-signal allocation in 120 s or less (relaxed from the 60 s final target for a v0 model), and E3 shows min-move moving fewer unchanged signals than from-scratch on at least 8 of 10 revision pairs.
- **Change method, not problem:** if E2 misses, decompose by cabinet and group and hint from greedy; recheck Wednesday 14 Oct. If E3 misses, pin unchanged cables, place new cables in free slots, and repair locally with CP-SAT on the affected CIOCs only.
- **Pitch change:** if greedy B1 is also checker-clean on all 20 adversarial files, lead the pitch with revisions and TQs rather than violation counts.

Questions to send the organisers tonight.

1. Which company set the industrial statements, and will its engineers judge them?
2. May code built in Stages 1–2 be used at the finale, and is internet available there?
3. May a 2-, 6- or 12-pair cable straddle two baseplates?
4. Does "in sequence" for an IO group mean consecutive slots or only ordered within the cabinet?
5. How is the baseplate's "CIOC no" derived from the CIOC name pattern?
6. When the data itself breaks a rule (A and B signals on one cable), is a TQ the expected output?
7. When will the sample MDB and allocation template be released?
8. What are the Stage 1 judging criteria and weights?

10. Final recommendation

Select Project IO SIGNAL SUDOKU and build CHARMSAT.

Decision	Answer
Problem statement	IO SIGNAL SUDOKU (industrial #02, IND p4–6)
Technical strategy	Cable blocks → rule compiler → OR-Tools CP-SAT with a lexicographic objective (fewest moves, then fewest cabinets); unsat-core TQs; an independent checker; Excel written into the organiser's template
Do not build	An LLM or ML model that places signals; multi-agent orchestration; a vector database; a React front end; 3D cabinet renders; cloud deployment; other DCS families
Single most important differentiator	Minimum-move Rev B with an impact report, the pain the PDF names first (p4), proven rule-clean by the independent checker
Minimum evidence for the final claim	One reproducible benchmark run: B1 greedy vs CHARMSAT on 50 synthetic + 20 adversarial files + the organiser sample, showing 0 violations everywhere, Rev B moves beside a from-scratch re-solve, and runtime at 10,000 signals
Switch to T4-2 if	Gate 1 fails on Saturday at 13:00 (E1 not checker-clean on 1,000 signals in 30 s)
After Stage 1	Change method, never problem: greedy + local CP-SAT repair, with the same checker

11. Sources and reproducibility

All accessed 9 Oct 2026. "Opened" = full page fetched; "excerpt" = read from search-result text only, so re-open before quoting it to judges.

Official competition material

- Attached PDFs: industrial challenges (IND, 8 pp.), iMocha CODE APEX 3.0 track (IMO, 9 pp.), Track 4 powered by UST (T4, 12 pp.). Primary.
- VIT CSE-AI Events and Highlights: CODE APEX 3.0 stages and prizes, CODEAPEX 2.0, Code Verse 2025 winners. Opened.

- CODE APEX 3.0 on Unstop. Did not render; check it from your logged-in account.

Company documentation

- Emerson DeltaV Electronic Marshalling and US page. Excerpt.
- iMocha Skills Assessment. Excerpt.
- CodeSignal: iMocha alternatives. Excerpt; competitor-written, secondary.
- Zeal Polytechnic blog: Emerson Talegaon visit. Excerpt; secondary.

Previous hackathon results and judging

- UST D3CODE 2025 winners, 15 Oct 2025. Opened.
- UST D3CODE 2024 launch and criteria, 16 Sep 2024. Excerpt.
- D3CODE 2024 submission guideline. Excerpt.
- SDSLabs on UST D3CODE 2019, Jan 2020. Excerpt; community-written.
- Emerson Innovation Challenge, Philippines, Nov 2019. Excerpt; analogous only.

Datasets

- eWINE UWB LOS/NLOS data set. Excerpt.
- UWB Positioning and Tracking Data Set, v2.0, 25 Aug 2023, CC BY 4.0. Excerpt.
- UWB ranging, three platforms, 12 Aug 2022. Excerpt.
- Industrial UWB TWR/TDOA dataset, DOI 10.21227/tr65-4795. Excerpt.
- Intel Berkeley Lab data, 2004. Excerpt.
- Benchmark datasets for sensor fault detection, 2016. Excerpt.
- PID2Graph (zenodo.org/records/14803338, per arXiv 2411.13929v3) and Dataset-P&ID / Digitize-PID. Excerpt.

Research papers

- Sharma, Golubchik, Govindan, Sensor faults: detection methods and prevalence, ACM TOSN 2010. Excerpt.
- Stürmer, Graumann, Koch, From Engineering Diagrams to Graphs, rev. 19 Dec 2025. Excerpt.
- SynthPID: P&ID digitization from topology-preserving synthetic data, 2026. Excerpt.
- Szajna et al., Deep CNN method for wire marking identification, Energies 2021. Excerpt.
- WebTestPilot, WebTestBench, Are Autonomous Web Agents Good Testers?. Excerpt.
- UTIL UWB TDOA dataset paper. Excerpt.

- EP 3605758 A1: testing control-cabinet contents against a plan, patent prior art for EAGLE EYE. Excerpt.

Open-source tools and evaluation references

- OR-Tools CP-SAT source and discussion #4110 on assumptions. Excerpt.
- CPMpy: unsat-core extraction and MUS tools. Excerpt.
- uwb_positioning analysis code, Apache 2.0 per the Zenodo record. Excerpt.